Shortest Path Problems

- How can we find the shortest route between two points on a road map?
- Model the problem as a graph problem:
  - Road map is a weighted graph:

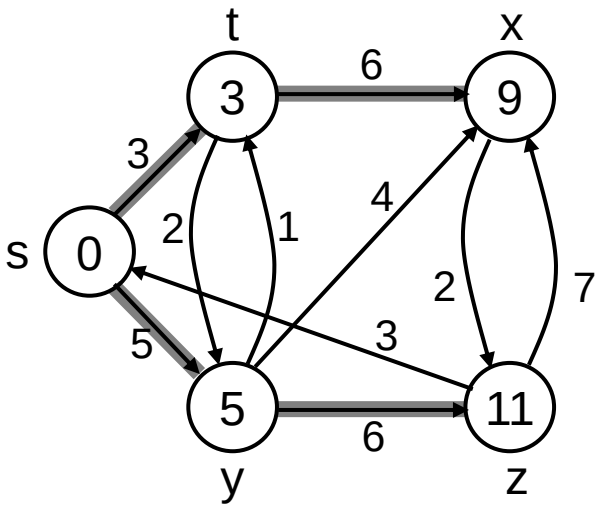
    vertices = cities

    edges = road segments between cities

    edge weights = road distances
  - Goal: find a shortest path between two vertices (cities)

Shortest Path Problem

- **Input:**
  - Directed graph G = (V, E)
  - Weight function w : E → **R**



- **Weight of path** $p = \langle v_0, v_1, \ldots, v_k \rangle$

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

- **Shortest-path weight** from u to v:

$$\delta(u, v) = \min \begin{cases} w(p) : u \overset{p}{\rightsquigarrow} v & \text{if there exists a path from } u \text{ to } v \\ \infty & \text{otherwise} \end{cases}$$

- **Note:** there might be multiple shortest paths from u to v

Variants of Shortest Path

- **Single-source shortest paths**
  - G = (V, E) $\Rightarrow$ find a shortest path from a given source vertex s to each vertex $v \in V$
- **Single-destination shortest paths**
  - Find a shortest path to a given destination vertex **t** from each vertex **v**
  - Reversing the direction of each edge $\Rightarrow$ single-source

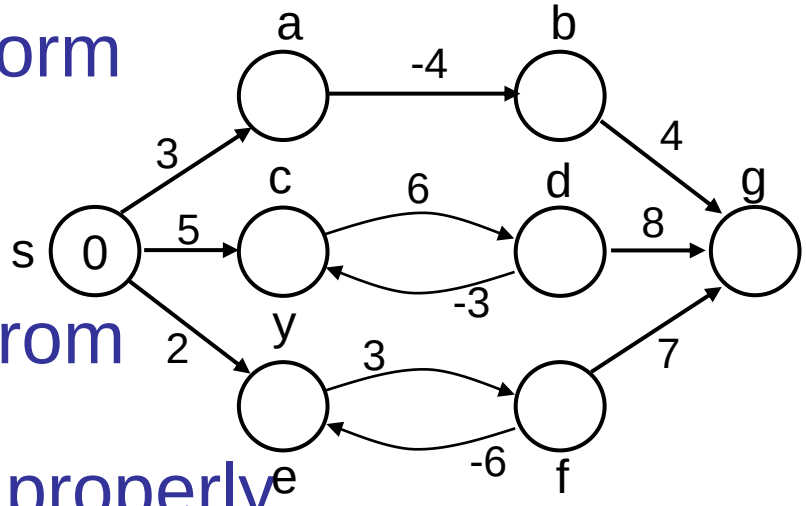
Variants of Shortest Paths (cont'd)

- **Single-pair shortest path**
  - Find a shortest path from u to v for given vertices u and v
- **All-pairs shortest-paths**
  - Find a shortest path from u to v for every pair of vertices u and v

Negative-Weight Edges

- Negative-weight edges may form negative-weight cycles
- If such cycles are reachable from the source, then δ(s, v) is not properly defined!
  - Keep going around the cycle, and get

    w(s, v) = - ∞ for all v on the cycle



Negative-Weight Edges

- $s \rightarrow a$: only one path

  $\delta(s, a) = w(s, a) = 3$

- $s \rightarrow b$: only one path

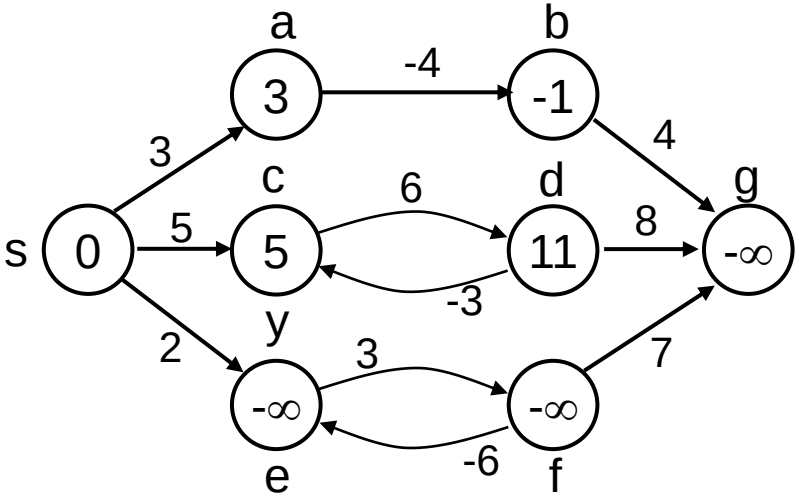
  $\delta(s, b) = w(s, a) + w(a, b) = -1$

- $s \rightarrow c$: infinitely many paths

  $\langle s, c \rangle$, $\langle s, c, d, c \rangle$, $\langle s, c, d, c, d, c \rangle$

  cycle has positive weight (6 - 3 = 3)

  $\langle s, c \rangle$ is shortest path with weight $\delta(s, c) = w(s, c) = 5$

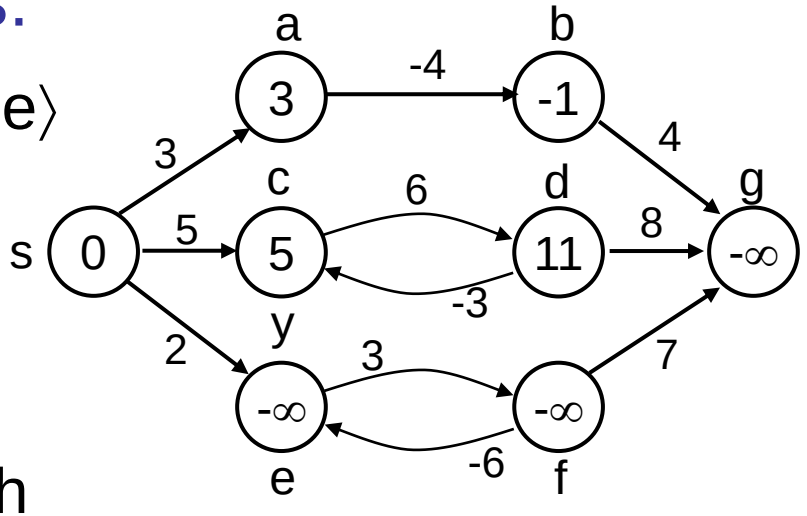


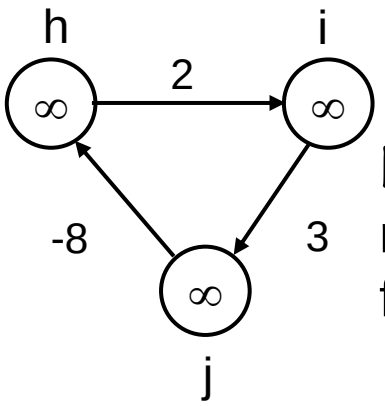
Negative-Weight Edges

- $s \to e$: infinitely many paths:
  - $\langle s, e\rangle$, $\langle s, e, f, e\rangle$, $\langle s, e, f, e, f, e\rangle$
  - cycle $\langle e, f, e\rangle$ has negative weight:

    $3 + (-6) = -3$
  - can find paths from s to e with arbitrarily large negative weights
  - $\delta(s, e) = -\infty \Rightarrow$ no shortest path exists between s and e
  - Similarly: $\delta(s, f) = -\infty$,

    $\delta(s, g) = -\infty$





h, i, j not reachable from s

$\delta(s, h) = \delta(s, i) = \delta(s, j) = \infty$

Cycles

- Can shortest paths contain cycles?
- Negative-weight cycles No!
  - Shortest path is not well defined
- Positive-weight cycles: No!
  - By removing the cycle, we can get a shorter path
- Zero-weight cycles
  - No reason to use them
  - Can remove them to obtain a path with same weight

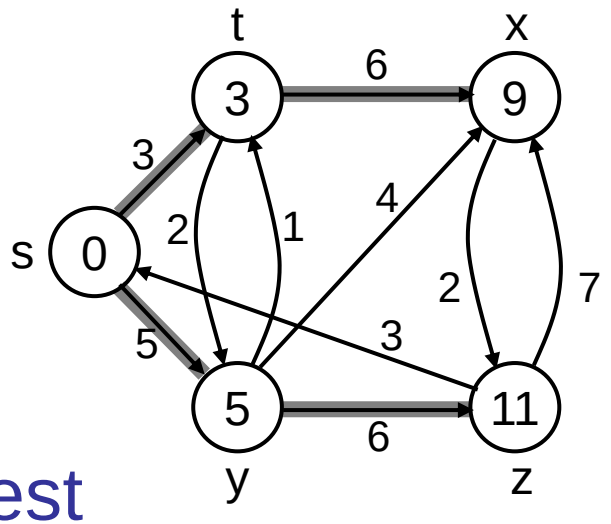
Algorithms

- Bellman-Ford algorithm
  - Negative weights are allowed
  - Negative cycles reachable from the source are not allowed.
- Dijkstra's algorithm
  - Negative weights are not allowed
- Operations common in both algorithms:
  - Initialization
  - Relaxation

Shortest-Paths Notation

For each vertex $v \in V$:

- $\delta(s, v)$: **shortest-path weight**
- d[v]: shortest-path weight **estimate**
  - Initially, $d[v]=\infty$
  - $d[v] \rightarrow \delta(s,v)$ as algorithm progresses
- $\pi[v]$ = **predecessor** of v on a shortest path from ***s***
  - If no predecessor, $\pi[v]$ = NIL
  - π induces a tree—**shortest-path tree**



Initialization

Alg.: INITIALIZE-SINGLE-SOURCE(V, s)

1. **for** each $v \in V$
2. **do** $d[v] \leftarrow \infty$
3. $\pi[v] \leftarrow$ NIL
4. $d[s] \leftarrow 0$

- All the shortest-paths algorithms start with INITIALIZE-SINGLE-SOURCE

Relaxation Step

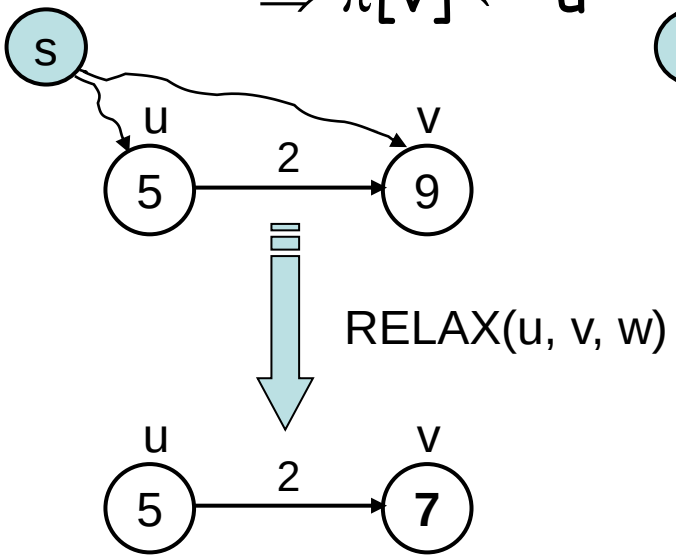
- **Relaxing** an edge (u, v) = testing whether we can improve the shortest path to v found so far by going through u

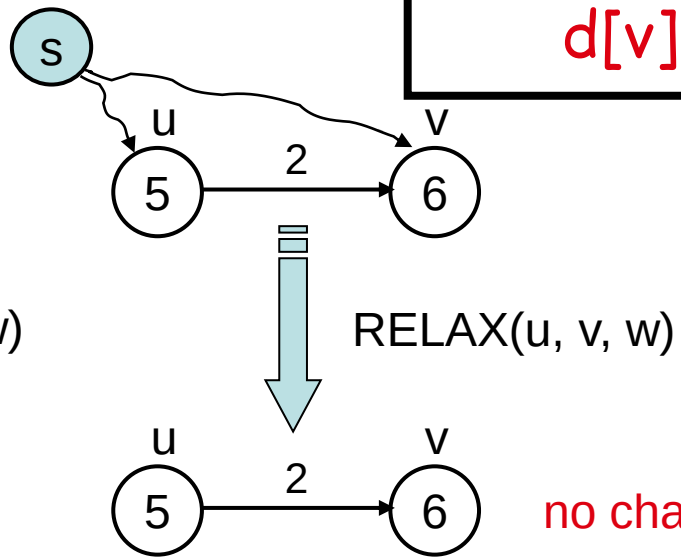
  If d[v] > d[u] + w(u, v)

  we can improve the shortest path to v

  $\Rightarrow$ d[v]=d[u]+w(u,v)

  $\Rightarrow$ π[v] $\leftarrow$ u





After relaxation:

$d[v] \leq d[u] + w(u, v)$

no change

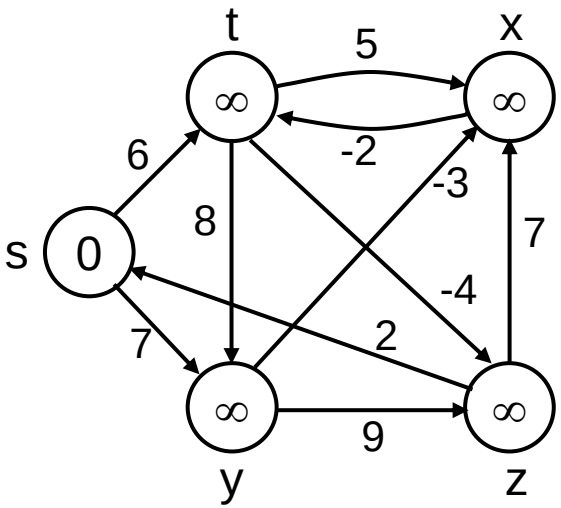
Bellman-Ford Algorithm

- Single-source shortest path problem
  - Computes $\delta(s, v)$ and $\pi[v]$ for all $v \in V$
- Allows negative edge weights - can detect negative cycles.
  - Returns TRUE if no negative-weight cycles are reachable from the source s
  - Returns FALSE otherwise $\Rightarrow$ no solution exists

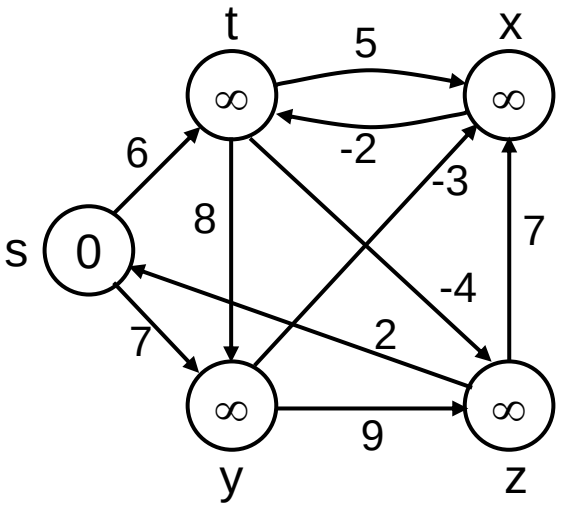
Bellman-Ford Algorithm (cont'd)

- Idea:
  - Each edge is relaxed |V–1| times by making |V-1| passes over the whole edge set.
  - To make sure that each edge is relaxed exactly |V – 1| times, it puts the edges in an unordered list and goes over the list |V – 1| times.

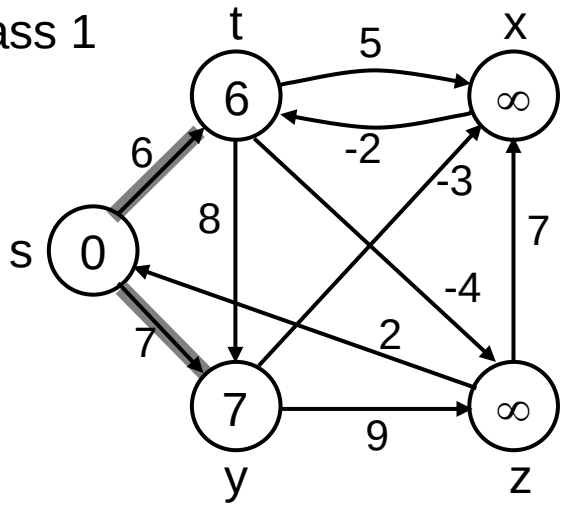
(t, x), (t, y), (t, z), (x, t), (y, x), (y, z), (z, x), (z, s), (s, t), (s, y)



BELLMAN-FORD(V, E, w, s)



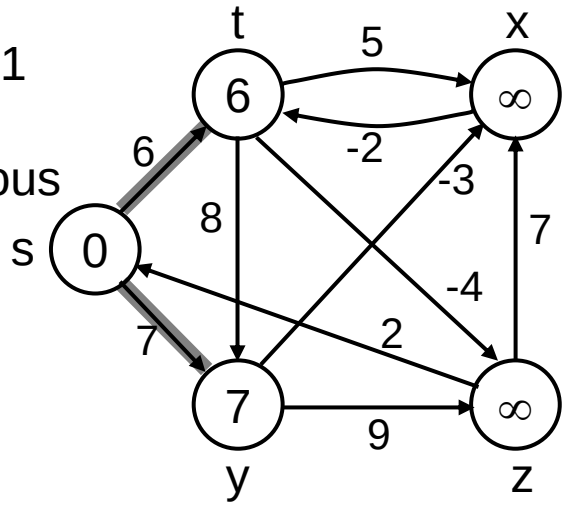
Pass 1



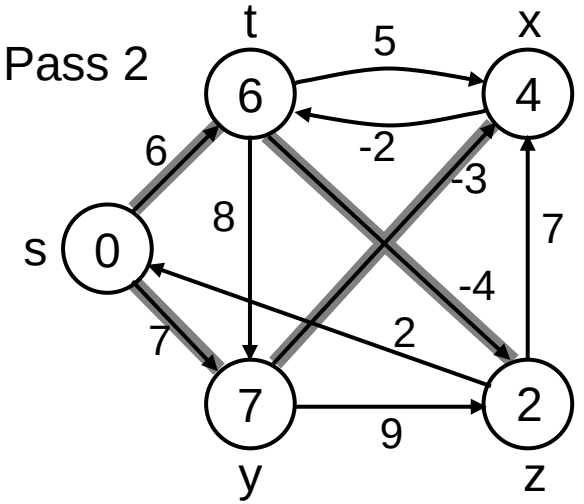
E: (t, x), (t, y), (t, z), (x, t), (y, x), (y, z), (z, x), (z, s), (s, t), (s, y)

Example (t, x), (t, y), (t, z), (x, t), (y, x), (y, z), (z, x), (z, s), (s, t), (s, y)

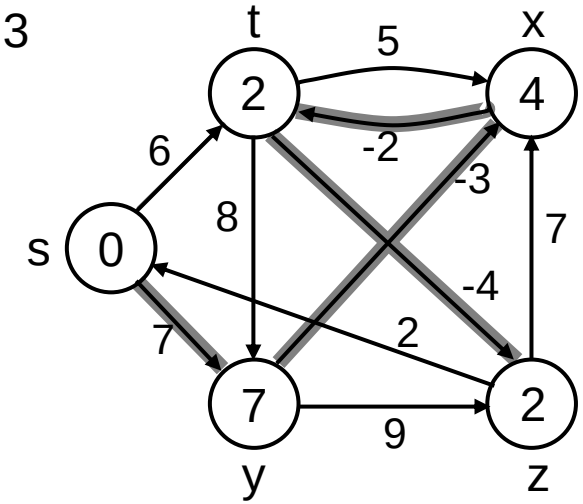
Pass 1 (from previous slide)



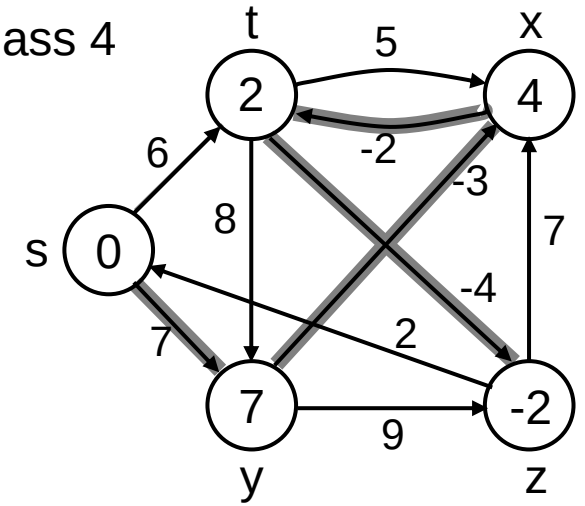
Pass 2



Pass 3

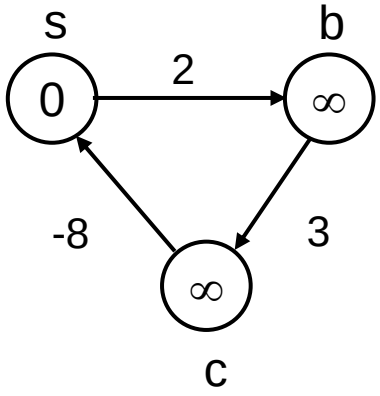


Pass 4

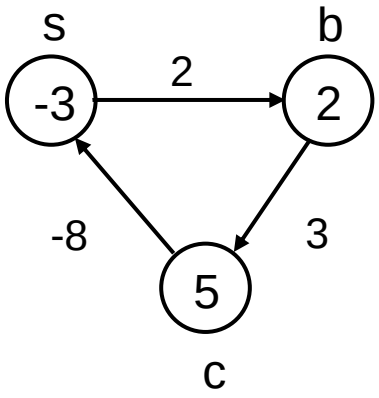


Detecting Negative Cycles (perform extra test after V-1 iterations)

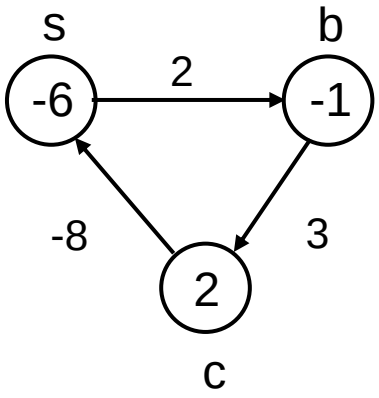
- **for** each edge (u, v) $\in$ E
- **do if** d[v] > d[u] + w(u, v)
- **then return** FALSE
- **return** TRUE



1st pass



2nd pass



(s,b) (b,c) (c,s)

Look at edge (s, b):

d[b] = -1
d[s] + w(s, b) = -4

$\Rightarrow$ d[b] > d[s] + w(s, b)

BELLMAN-FORD(V, E, w, s)

```
1.  INITIALIZE-SINGLE-SOURCE(V, s)                ← Θ(V)
2.  for i ← 1 to |V| - 1                          ← O(V)
3.      do for each edge (u, v) ∈ E               ← O(E)      } O(VE)
4.          do RELAX(u, v, w)
5.  for each edge (u, v) ∈ E                      ← O(E)
6.      do if d[v] > d[u] + w(u, v)
7.          then return FALSE
8.  return TRUE
```

Running time: O(V+VE+E)=O(VE)